

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО

ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т

по лабораторной работе по предмету Проектирование

и реализация баз данных №6

«Работа с БД в СУБД MongoDB»

Обучающийся: Сусликова Вероника Денисовна, К3240

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург,

2026

Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание

Последовательно выполнить все практические задания

Выполнение работы

Практическое задание 2.1.1:

- 1) Создайте базу данных learn.
- 2) Заполните коллекцию единорогов unicorns
- 3) Используя 2-ой способ, вставьте в коллекцию документ
- 4) Проверьте содержимое коллекции с помощью метода find

```
test> use learn
switched to db learn
learn> db.unicorns.insertOne({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
{
  acknowledged: true,
  insertedId: ObjectId('6a3e503c834161b1083682d2')
}
learn> db.unicorns.insertOne({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
| db.unicorns.insertOne({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
| db.unicorns.insertOne({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
| db.unicorns.insertOne({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
|
| db.unicorns.insertOne({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
| db.unicorns.insertOne({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
| db.unicorns.insertOne({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
| db.unicorns.insertOne({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
| db.unicorns.insertOne({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
| db.unicorns.insertOne({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
|
{
  acknowledged: true,
  insertedId: ObjectId('6a3e5094834161b1083682dc')
}
```

Рисунок 1.1 - выполнение пунктов 1 и 2 задания

```
learn> document={name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
| db.unicorns.insert(document)
|
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a3e5b51834161b1083682dd') }
}
```

Рисунок 1.2 - выполнение пункта 3 задания

```

{
  _id: ObjectId('6a3e5094834161b1083682dc'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a3e5b51834161b1083682dd'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}

```

Рисунок 1.3 - выполнение пункта 4 задания (последние 2 записи)

Практическое задание 2.2.1:

- 1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.
- 2) Найдите всех самцов, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

```

learn> db.unicorns.find({gender: "m"}).sort({name: 1})
[
  {
    _id: ObjectId('6a3e5b51834161b1083682dd'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a3e5005834161b1083682d1'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a3e503c834161b1083682d2'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
]

```

Рисунок 2.1 - выполнение пункта 1 задания - сортировка самцов (первые 3 пункта ответа, без ограничения)

```
learn> db.unicorns.find({gender: "f"}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a3e5094834161b1083682d3'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a3e5094834161b1083682d7'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a3e5094834161b1083682da'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> |
```

Рисунок 2.2 - выполнение пункта 1 задания - сортировка самок

```
learn> db.unicorns.find({gender: "m", loves: "carrot"}).limit(1)
[
  {
    _id: ObjectId('6a3e5005834161b1083682d1'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  }
]
learn> db.unicorns.findOne({gender: "m", loves: "carrot"})
{
  _id: ObjectId('6a3e5005834161b1083682d1'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
learn> |
```

Рисунок 2.3 - выполнение пункта 2 задания - результаты для limit(1) и findOne одинаковы, как и должны быть

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find({gender: "m"}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a3e5005834161b1083682d1'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a3e503c834161b1083682d2'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a3e5094834161b1083682d4'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
]
```

Рисунок 3 - выполнение 3 задания

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({ $natural: -1 })
[
  {
    _id: ObjectId('6a3e5b51834161b1083682dd'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a3e5094834161b1083682dc'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a3e5094834161b1083682db'),
    name: 'Pilot',

```

Рисунок 4 - выполнение 4 задания

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {loves: {$slice : [0, 1]}, _id: 0})
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,

```

Рисунок 5 - выполнение 5 задания

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: "f", weight: {$gt : 500, $lt: 700}}, {_id:0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

Рисунок 6 - выполнение 6 задания

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: "m", weight: {$gt : 500}, loves: {$all : ["grape", "lemon"]}}, {_id:0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> |
```

Рисунок 7 - выполнение 7 задания

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires: {$exists:0}})
[
  {
    _id: ObjectId('6a3e5094834161b1083682dc'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

Рисунок 8 - выполнение 8 задания

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({gender: "m"},{loves: {$slice : [0, 1]}, name: 1}).sort({name: 1})
[
  {
    _id: ObjectId('6a3e5b51834161b1083682dd'),
    name: 'Dunx',
    loves: [ 'grape' ]
  },
  {
    _id: ObjectId('6a3e5005834161b1083682d1'),
    name: 'Horny',
    loves: [ 'carrot' ]
  },
  {
    _id: ObjectId('6a3e503c834161b1083682d2'),
    name: 'Horny',
    loves: [ 'carrot' ]
  },
  {
    _id: ObjectId('6a3e5094834161b1083682d8'),
    name: 'Kenny',
    loves: [ 'grape' ]
  }
]
```

Рисунок 9 - выполнение 9 задания

Практическое задание 3.1.1:

- 1) Создайте коллекцию towns, включающую документы
- 2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.
- 3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
learn> db.towns.insertOne({name: "Punxsutawney ",
|   populatiuon: 6200,
|   last_sensus: ISODate("2008-01-31"),
|   famous_for: [""],
|   mayor: {
|     name: "Jim Wehrle"
|   }})
|
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a3f5b8504bf1d5caa3682d1')
| }
learn> db.towns.insertOne({name: "New York",
|   populatiuon: 22200000,
|   last_sensus: ISODate("2009-07-31"),
|   famous_for: ["status of liberty", "food"],
|   mayor: {
|     name: "Michael Bloomberg",
|     party: "I"}})
| db.towns.insertOne({name: "Portland",
|   populatiuon: 528000,
|   last_sensus: ISODate("2009-07-20"),
|   famous_for: ["beer", "food"],
|   mayor: {
|     name: "Sam Adams",
|     party: "D"}})
|
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a3f5b8e04bf1d5caa3682d3')
| }
```

Рисунок 10.1 - выполнение пункта 1 задания 10 вставка данных

```
learn> db.towns.find({"mayor.party": "I"},{mayor:1, name: 1})
[
  {
    _id: ObjectId('6a3f5b8e04bf1d5caa3682d2'),
    name: 'New York',
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn> |
```

Рисунок 10.2 - выполнение пункта 2 задания 10

```
learn> db.towns.find({"mayor.party":{"$exists:0}},{mayor:1, name: 1})
[
  {
    _id: ObjectId('6a3f5b8504bf1d5caa3682d1'),
    name: 'Punxsutawney ',
    mayor: { name: 'Jim Wehrle' }
  }
]
learn> |
```

Рисунок 10.3 - выполнение пункта 3 задания 10

Практическое задание 3.1.2:

- 1) Сформировать функцию для вывода списка самцов единорогов.
- 2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
- 3) Вывести результат, используя forEach

```
learn> var cursor = db.unicorns.find({gender:"m"});null;
| cursor.sort({name:1}).limit(2);null;
| cursor.forEach(function(obj){print(obj);})
|
| {
|   _id: ObjectId('6a3e5b51834161b1083682dd'),
|   name: 'Dunx',
|   loves: [ 'grape', 'watermelon' ],
|   weight: 704,
|   gender: 'm',
|   vampires: 165
| }
| {
|   _id: ObjectId('6a3e5005834161b1083682d1'),
|   name: 'Horny',
|   loves: [ 'carrot', 'papaya' ],
|   weight: 600,
|   gender: 'm',
|   vampires: 63
| }
learn> |
```

Рисунок 11 - выполнение задания 11

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({gender: "f", weight: {$gt:500, $lt:600}}).count()  
2  
learn>
```

Рисунок 12 - выполнение задания 12

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")  
[  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]  
learn>
```

Рисунок 13 - выполнение задания 13

Практическое задание 3.2.3:

Подсчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.find().count()  
13  
learn> |
```

Рисунок 14 - выполнение задания 14

Практическое задание 3.3.1:

- 1) Выполнить команду
- 2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
TypeError: db.unicorns.save is not a function
learn> mongo --quiet
```

Рисунок 15 - выполнение задания 15

Дело в том, что функция `save` удалена и больше не может быть использована в новых версиях `mongo`. Раньше, однако, после выполнения команды у нас бы появилась запись (документ) о новом единороге.

Практическое задание 3.3.2:

- 1) Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
- 2) Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.updateOne({name : "Ayna"}, {$set: {weight : 800, vampires : 51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: "Ayna"})
[
  {
    _id: ObjectId('6a3e5094834161b1083682d7'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>
```

Рисунок 16 - выполнение задания 16

Практическое задание 3.3.3:

- 1) Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
- 2) Проверить содержимое коллекции `unicorns`.

- 1) Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
- 2) Проверить содержимое коллекции towns.

```
learn> db.towns.update({name : "Portland"}, {$unset: {"mayor.party" : 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name:"Portland"})
[
  {
    _id: ObjectId('6a3f5b8e04bf1d5caa3682d3'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensu: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> |
```

Рисунок 19 - выполнение задания 19

Практическое задание 3.3.6:

- 1) Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.
- 2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.update({name : "Pilot"}, {$push: {loves : "шоколад"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: "Pilot"})
[
  {
    _id: ObjectId('6a3e5094834161b1083682db'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'шоколад' ],
    weight: 650,
    gender: 'm',
    vampires: 64
  }
]
learn>
```

Рисунок 20 - выполнение задания 20

Практическое задание 3.3.7:

- 1) Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
- 2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.update({name : "Aurora"}, {$addToSet: {loves: {$each: ["сахар", "лимон"]}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: "Aurora"})
[
  {
    _id: ObjectId('6a3e5094834161b1083682d3'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'сахар', 'лимон' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
  }
]
```

Рисунок 21 - выполнение задания 21

Практическое задание 3.4.1:

- 1) Удалите документы с беспартийными мэрами.
- 2) Проверьте содержание коллекции.
- 3) Очистите коллекцию.
- 4) Просмотрите список доступных коллекций.

```
learn> db.towns.deleteMany({"mayor.party": {$exists:0}})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a3f5b8e04b1d5caa3682d2'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a40caa789ae2865063682d2'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> |
```

Рисунок 22.1 - выполнение пунктов 1 и 2 задания 22

```

learn> db.users.deleteMany({})
{ acknowledged: true, deletedCount: 0 }
learn> show collections
|
towns
unicorns
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
learn> db.towns.find()

learn> |

```

Рисунок 22.2 - выполнение пунктов 3 и 4 задания 22 - как видим после очищения коллекция все еще существует но пуста

Практическое задание 4.1.1:

- 1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.
- 2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
- 3) Проверьте содержание коллекции единорогов.

Запросы для выполнения задания 23 пунктов 1 и 2:

```
db.area.insertOne({_id:"zone1", name:"full_name_1",
description:"description1"})
```

```
db.area.insertOne({_id:"zone2", name:"full_name_2",
description:"description2"})
```

```
db.area.insertOne({_id:"zone3", name:"full_name_3",
description:"description3"})
```

```
db.unicorns.updateMany({gender: "f", name:"Ayna"},{$set: {area: {$ref:"areas",
$bid: "zone_1"}}})
```

```
db.unicorns.updateMany({gender: "f",  
name:"Aurora"},{$set:{area:{$ref:"areas", $id: "zone_2"}}})
```

```
db.unicorns.updateMany({gender: "f",  
name:"Solnara"},{$set:{area:{$ref:"areas", $id: "zone_3"}}})
```

```
{  
  _id: ObjectId('6a3e5094834161b1083682d3'),  
  name: 'Aurora',  
  loves: [ 'carrot', 'grape', 'сахар', 'лимон' ],  
  weight: 450,  
  gender: 'f',  
  vampires: 43,  
  languages: [ 'сахар', 'лимон' ],  
  area: DBRef('areas', 'zone_2')  
},  
{  
  _id: ObjectId('6a3e5094834161b1083682d6'),  
  name: 'Solnara',  
  loves: [ 'apple', 'carrot', 'chocolate' ],  
  weight: 550,  
  gender: 'f',  
  vampires: 80,  
  area: DBRef('areas', 'zone_3')  
},  
{  
  _id: ObjectId('6a3e5094834161b1083682d7'),  
  name: 'Ayna',  
  loves: [ 'strawberry', 'lemon' ],  
  weight: 800,  
  gender: 'f',  
  vampires: 51,  
  area: DBRef('areas', 'zone_1')  
},  
}
```

Рисунок 23 - выполнение задания 23

Практическое задание 4.2.1:

Проверьте, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique.

```
learn> db.unicorns.ensureIndex({name: 1}, {unique: true})  
MongoServerError[DuplicateKey]: Index build failed: 1414822a-3fa0-45b5-921c-3bc44b50d35f: Collection learn.unicorns ( 99  
e468c4-d09f-432f-8914-e04064a01aa7 ) :: caused by :: E11000 duplicate key error collection: learn.unicorns index: name_1  
dup key: { name: "Horny" }  
learn> db.unicorns.deleteOne({name:"Horny"})  
{ acknowledged: true, deletedCount: 1 }  
learn> db.unicorns.ensureIndex({name: 1}, {unique: true})  
[ 'name_1' ]  
learn> |
```

Рисунок 24 - выполнение задания 24

Пояснение: при создании базы я нечаянно создала две записи одинакового единорога, поэтому уникальный индекс и не мог создаться - после удаления одной из копий индекс прекрасно создался, как и должен был по заданию.

Практическое задание 4.3.1:

- 1) Получите информацию о всех индексах коллекции unicorns .
- 2) Удалите все индексы, кроме индекса для идентификатора.
- 3) Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex("_id")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> |
```

Рисунок 25 - выполнение задания 25 - индекс идентификатора невозможно удалить

Практическое задание 4.4.1:

- 1) Создайте объемную коллекцию numbers, задействовав курсор:
`for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}`
- 2) Выберите последних четыре документа.
- 3) Проанализируйте план выполнения запроса. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)
- 4) Создайте индекс для ключа value.
- 5) Получите информацию о всех индексах коллекции numbers.
- 6) Выполните запрос 2.
- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

```
learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a40d3ee89ae286506380972') }
}
```

Рисунок 26.1 - выполнение задания 26 пункта 1

```
learn> db.numbers.explain("executionStats").find({}).sort({value:-1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 8,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 119,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 92,
      works: 100006,

```

Рисунок 26.2 - выполнение задания 26 пунктов 2 и 3 - время выполнения запроса 119 миллисекунд без индекса

```

learn> db.numbers.ensureIndex({"value":1})
[ 'value_1' ]
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> |

```

Рисунок 26.3 - выполнение задания 26 пунктов 4 и 5

```

learn> db.numbers.explain("executionStats").find({}).sort({value:-1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 5,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  }
}

```

Рисунок 26.4 - выполнение задания 26 пункта 6

```
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 8,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 4
  }
}
```

Рисунок 26.5 - выполнение задания 26 пунктов 7 и 8 - время выполнения запроса с индексом 8 миллисекунд, что значительно меньше, чем без него, что доказывает эффективность индексов на больших объемах данных в БД

Вывод

Мы выполнили нашу цель овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.